

EXP 05

FCFS, SJF, Priority & Round Robin Scheduling

AIM:

To implement First-come First-serve (FCFS) scheduling technique, Shortest Job First(SJF) scheduling technique, priority scheduling technique and Round Robin (RR) scheduling technique

1. FCFS Scheduling

ALGORITHM

1. Start
2. Get number of processes
3. Get burst time for each process
4. Set waiting time of first process as 0
5. Calculate waiting time of remaining processes
6. Calculate turnaround time for each process
7. Display process details
8. Calculate average waiting time and average turnaround time
9. Display averages
10. End

CODE:

```
#include <stdio.h>

int main() {
    int n, i;
    int bt[20], wt[20], tat[20];
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
```

```

printf("Enter burst time for each process:\n");
for(i = 0; i < n; i++) {
    printf("P%d: ", i + 1);
    scanf("%d", &bt[i]);
}

wt[0] = 0;

for(i = 1; i < n; i++) {
    wt[i] = wt[i - 1] + bt[i - 1];
}

for(i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
    avg_wt += wt[i];
    avg_tat += tat[i];
}

printf("\nProcess\tBT\tWT\tTAT\n");
for(i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
}

printf("\nAverage Waiting Time = %.2f", avg_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat / n);

return 0;
}

```

2. SJF Scheduling

ALGORITHM

1. Start
2. Get number of processes
3. Get burst time for each process
4. Sort processes based on shortest burst time
5. Set waiting time of first process as 0
6. Calculate waiting time for remaining processes
7. Calculate turnaround time
8. Display process details
9. Calculate average waiting time and turnaround time
10. End

CODE:

```
#include <stdio.h>

int main() {
    int n, i, j, temp;
    int bt[20], wt[20], tat[20], p[20];
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        p[i] = i + 1;
        printf("Enter burst time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    for(i = 0; i < n - 1; i++) {
        for(j = i + 1; j < n; j++) {
            if(bt[i] > bt[j]) {
                temp = bt[i];
                bt[i] = bt[j];
            }
        }
    }

    wt[0] = 0;
    for(i = 1; i < n; i++) {
        wt[i] = wt[i-1] + bt[i-1];
    }

    for(i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    avg_wt = (wt[n-1] + 1) / n;
    avg_tat = (tat[n-1] + 1) / n;

    printf("Average waiting time: %.2f\n", avg_wt);
    printf("Average turnaround time: %.2f\n", avg_tat);
}
```

```

        bt[j] = temp;

        temp = p[i];
        p[i] = p[j];
        p[j] = temp;
    }
}

wt[0] = 0;

for(i = 1; i < n; i++) {
    wt[i] = wt[i - 1] + bt[i - 1];
}

for(i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
    avg_wt += wt[i];
    avg_tat += tat[i];
}

printf("\nProcess\tBT\tWT\tTAT\n");
for(i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\n", p[i], bt[i], wt[i], tat[i]);
}

printf("\nAverage Waiting Time = %.2f", avg_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat / n);

return 0;
}

```

3. Priority Scheduling ALGORITHM

1. Start

2. Get number of processes
3. Get burst time and priority for each process
4. Sort processes based on priority
5. Lower priority number means higher priority
6. Set waiting time of first process as 0
7. Calculate waiting time
8. Calculate turnaround time
9. Display process details and averages
10. End

CODE

```
#include <stdio.h>

int main() {
    int n, i, j, temp;
    int bt[20], wt[20], tat[20], p[20], pr[20];
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        p[i] = i + 1;
        printf("Enter burst time and priority for P%d: ", i + 1);
        scanf("%d %d", &bt[i], &pr[i]);
    }

    for(i = 0; i < n - 1; i++) {
        for(j = i + 1; j < n; j++) {
            if(pr[i] > pr[j]) {
                temp = pr[i];
                pr[i] = pr[j];
                pr[j] = temp;

                temp = bt[i];
                bt[i] = bt[j];
```

```

        bt[j] = temp;

        temp = p[i];
        p[i] = p[j];
        p[j] = temp;
    }
}

wt[0] = 0;

for(i = 1; i < n; i++) {
    wt[i] = wt[i - 1] + bt[i - 1];
}

for(i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
    avg_wt += wt[i];
    avg_tat += tat[i];
}

printf("\nProcess\tBT\tPriority\tWT\tTAT\n");
for(i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t\t%d\t%d\n", p[i], bt[i], pr[i], wt[i],
tat[i]);
}

printf("\nAverage Waiting Time = %.2f", avg_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat / n);

return 0;
}

```

4. Round Robin Scheduling ALGORITHM

1. Start
2. Get number of processes
3. Get burst time for each process
4. Get time quantum
5. Store burst time in remaining time array
6. Execute each process for one time quantum
7. If process is not completed, continue in next round
8. If process completes, calculate waiting time
9. Calculate turnaround time
10. Display process details and averages
11. End

CODE:

```
#include <stdio.h>

int main() {
    int n, i, time = 0, tq, completed = 0;
    int bt[20], rt[20], wt[20], tat[20];
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        printf("Enter burst time for P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }

    printf("Enter time quantum: ");
    scanf("%d", &tq);

    while(completed < n) {
        for(i = 0; i < n; i++) {
```

```

        if(rt[i] > 0) {
            if(rt[i] > tq) {
                time += tq;
                rt[i] -= tq;
            } else {
                time += rt[i];
                wt[i] = time - bt[i];
                rt[i] = 0;
                completed++;
            }
        }
    }
}

for(i = 0; i < n; i++) {
    tat[i] = bt[i] + wt[i];
    avg_wt += wt[i];
    avg_tat += tat[i];
}

printf("\nProcess\tBT\tWT\tTAT\n");
for(i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
}

printf("\nAverage Waiting Time = %.2f", avg_wt / n);
printf("\nAverage Turnaround Time = %.2f\n", avg_tat / n);

return 0;
}

```

RESULT:

Thus, FCFS, SJF, Priority and Round Robin CPU scheduling algorithms were successfully implemented and executed.